File Handling

CSL-100

November 18. 2024

Today

- File Processing
- Multi File Programming
- Makefile Utility

Files

- What is a file?
  - Collection of bytes stored on secondary storage like hard disks (not RAM).
- Any *addressable* part of the file system in an Operating system can be a file.
  - includes such strange things as /dev/null (nothing), /dev/usb (USB port), /dev/audio (speakers)

File Access

- 3 files are always connected to a C program :
  - stdin : the standard input, from where scanf, getchar(), gets() etc. read input from
  - stdout : the standard output, to where printf(), putchar(), puts() etc. output to.
  - stderr : standard error console.

File Handling in C

1. Open the file for reading/writing etc.: fopen
   - return a *file pointer*
   - pointer points to an internal structure containing information about the file:
     - location of a file
     - the current position being read in the file, etc.

   FILE* fopen (char *name, char *mode)
2. Read/Write to the file

   int fscanf(FILE *fp, char *fo

   int fprintf(FILE *fp, char *forma

   int fputs(const char* str, FILE *f
3. Close the File.

   int fclose(FILE *fp)

Compared to scanf and printf – a new (first) argument fp is added

Opening Files

FILE* fopen (char *name, char *mode)

- The first argument is the name of the file
  - can be given in short form (e.g. “inputfile”) or the full path name (e.g. “/home/don/inputfile”)
- The second argument is the mode in which we want to open the file. Common modes include:
  - “r” : read-only. Any write to the file will fail. File must exist.
  - “w” : write. The first write happens at the beginning of the file, by default. Thus, may overwrite the current content. A new file is created if it does not exist.
  - “a” : append. The first write is to the end of the current content. File is created if it does not exist.

Opening Files

- If successful, fopen returns a *file pointer* – this is later used for fprintf, fscanf etc.
- If unsuccessful, fopen returns a NULL.
- It is a good idea to check for errors (e.g. Opening a file on a CDROM using “w” mode etc.)

Closing Files

- An open file must be closed after last use
  - allows reuse of FILE* resources
  - flushing of ***buffered*** data (to actually write!)

File I/O: Example

- Write a program that will take two filenames, and print contents to the standard output. The contents of the first file should be printed first, and then the contents of the second.
- The algorithm:
  1. Read the file names.
  2. Open file 1. If open failed, we exit
  3. Print the contents of file 1 to stdout
  4. Close file 1
  5. Open file 2. If open failed, we exit
  6. Print the contents of file 2 to stdout
  7. Close file 2

```
int main()
{
        FILE *fp;  char filename1[128], filename2[128];
        scanf(“%s”, filename1);
        scanf(“%s”, filename2);
            fp = fopen( filename1, "r" );
        if(fp == NULL) {
              fprintf(stderr, "Opening File %s failed\n",
filename1);
              return -1;
        }
        copy_file(fp, stdout);
        fclose(fp);
        fp = fopen( filename2, "r" );
        if (fp == NULL) {
              fprintf(stderr, "Opening File %s failed\n",
filename2);
              return -1;
        }
        copy_file (fp, stdout);
        fclose(fp);
        return 0;
}
```

The Program: copy_file

```
void copy_file(FILE *fromfp, FILE
*tofp)
{
        char ch;

        while ( !feof ( fromfp ) ) {
          fscanf  ( fromfp, "%c", &ch );
          fprintf ( tofp, "%c", ch );
        }
}
```

File I/O: Example

- Write a program that will take two filenames (In.txt and Out.txt), and Copy the contents to the of In.txt to Out.txt.
- The algorithm:
  1. Read the file names. (In.txt and Out.txt)
  2. Open In.txt in read mod and Out.txt in write mode. If open failed, we exit
  3. Print the contents of In.txt to Out.txt
  4. Close In.txt and Out.txt

Some Other File Handling Functions

- **`int feof ( FILE* fp );`**
  - Checks whether the EOF is set for fp – that is, the EOF has been encountered. If EOF is set, it returns nonzero. Otherwise, returns 0.
- **`int ferror ( FILE *fp );`**
  - Checks whether the error indicator has been set for fp. (for example, write errors to the file.)

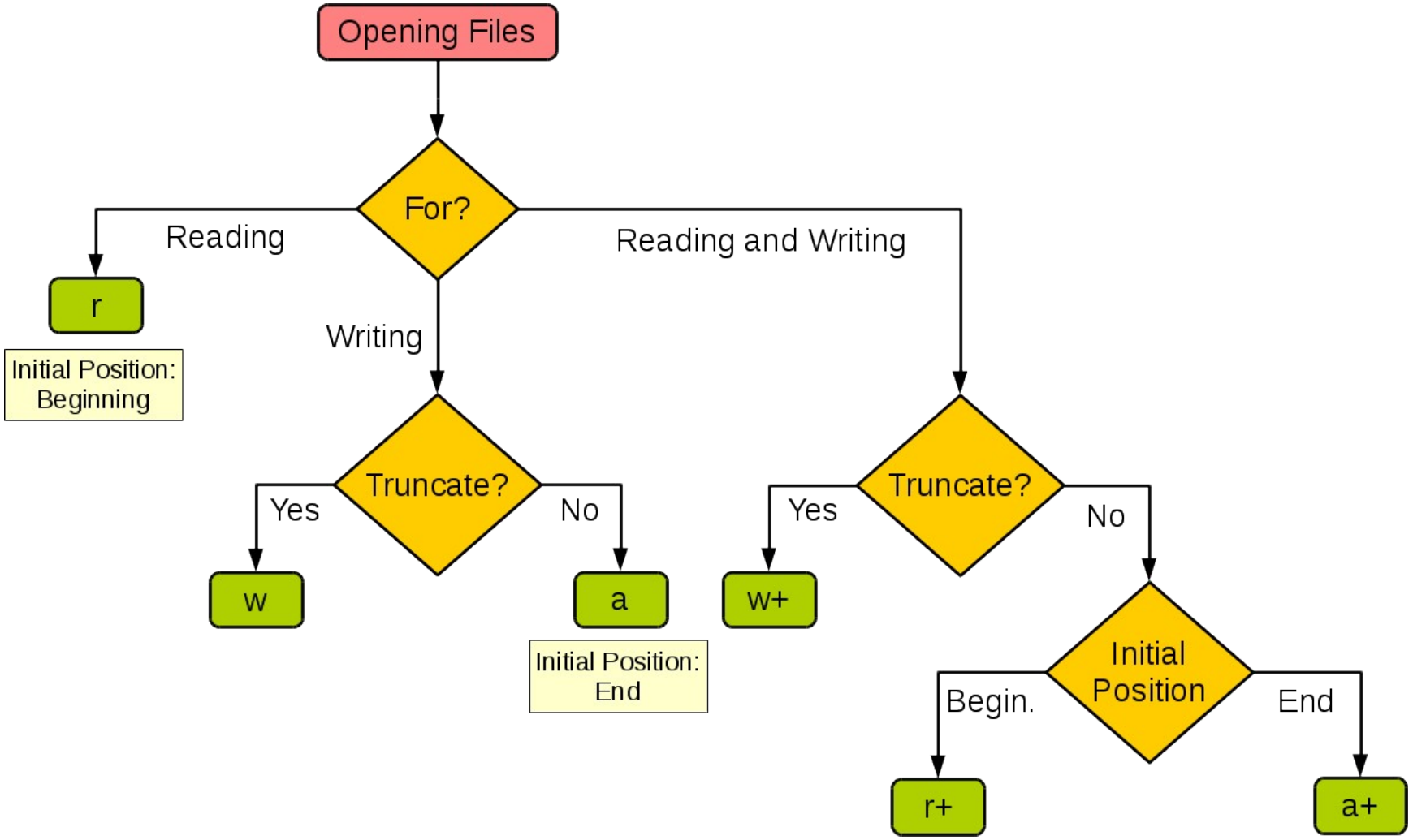
Some Other File Handling Functions

- **`int fseek(FILE *fp, long int offset, int origin);`**
  - To set the current position associated with fp, to a new position = origin + offset.
  - Origin can be:
    - SEEK_SET: beginning of file
    - SEEK_CURR: current position of file pointer
    - SEEK_END: End of file
  - Offset is the number of bytes.
- **`int ftell(FILE *fp)`**
  - Returns the current value of the position indicator of the stream.

Opening Files: More Modes

- There are other modes for opening files, as well.
  - “r+” : open a file for read and update. The file must be present.
  - “w+” : write/read. Create an empty file or overwrite an existing one.
  - “a+” : append/read. File is created if it doesn’t exist. The file position for reading is at the beginning, but output is appended to the end.

Opening Files: More Modes



File I/O Example

```
#include <stdio.h>
int main () {
        FILE * fp = fopen("file.txt","w+");
        fputs("This is C Programming Language", fp);
        fseek( fp, 7, SEEK_SET );
        fputs(" Java Programming Language", fp);
        fclose(fp);

        int c;
        fp = fopen("file.txt","r");
        while(1)  {
                c = fgetc(fp);
                if( feof(fp) ) break;
                printf("%c", c);
        }
        fclose(fp);
        return 0;
}
```

This is Java Programming Language

File I/O: stdout vs stderr

- What is the output of following program when run on a terminal:

```
#include <stdio.h>
int main()
{
  int input;
  scanf("%d", &input);
  fprintf(stdout, "Printing to STDOUT %d\n",
input);
  fprintf(stderr, "Printing to STDERR %d\n", input);
  return 0;
}
```

```
INPU
T
5
```

```
Printing to STDOUT 5
Printing to STDERR 5
```

An Exercise

- Write a C program to create class student database

1. Create student.doc file.
2. Add student data 1 by 1
3. Read from student.doc and display to user

Multi-File Program

```
#include <stdio.h>
void hello () {
  printf("Hello World!!\n");
}
int factorial(int n){
    if(n!=1){
        return(n * factorial(n-1));
    }
    else return 1;
}
int main(){
    hello();
    printf("The factorial of 5 is %d\n",
factorial(5));
    return 0;
}
```

Multi-File Program

```
#include<stdio.h>

void hello () {
  printf("Hello World!!\
n");
}
```

Hello.c

```
int factorial(int n){
    if(n!=1){
        return(n *
factorial(n-1));
    }
    else return 1;
```

fact.c

Multi-File Program

```
#include <stdio.h>
#include "fact.h"
#include "hello.h"

int main(){
    hello();
    printf("The factorial of 5 is %d\n",
factorial(5));
    return 0;
}
```

main.c

```
void hello ();
```

Hello.h

```
int factorial(int );
```

fact.h

How to Compile?

hello.c
hello.h

fact.c
fact.h

main.c

```
gcc –o out  hello.c fact.c
main.c

gcc –c hello.c    // Produces
hello.o
gcc –c fact.c
gcc –c main.c
gcc –o out hello.o fact.o
main.o
```

Command Line Args in C

- Write a program to read a name from command line, and say “Hello” to it.
- Some Example Interaction (Output in red):

```
$ ./a.out Subidh
Hello Subidh
$ ./a.out World
Hello World
$ ./a.out IC-100
Hello IC-100
```

Note: program has no sense of what is a name. It just prints the argument provided.

Command Line Args -> Args to main

- So far we used the following signature for main

  int main()

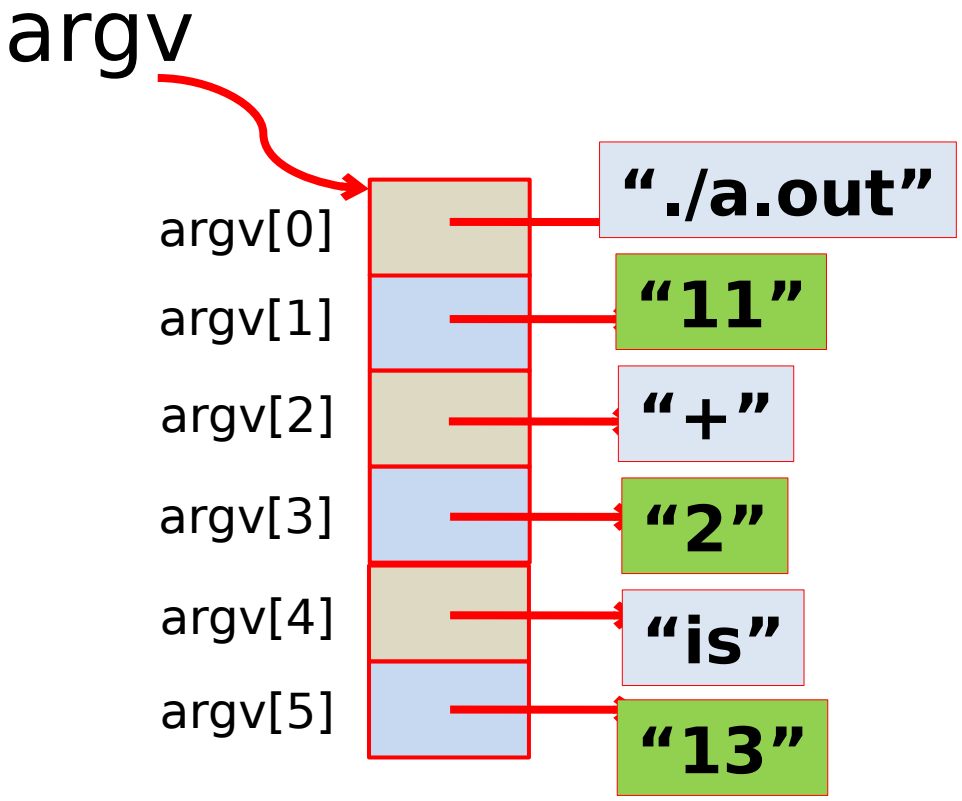
- But main can take arguments. The modified prototype of main is

  int main(int argc, char **argv)

  - Argument Count (argc): An int that tells the number of arguments passed on command line
  - Argument Values (argv): Array of strings. argv[i] is the i-th argument as string.

Args to main

- “./a.out” is included in arguments.
- ./a.out 11 + 2 is 13



Note that everything is treated as string, even the numbers!

Example

NOTE: char **argv is same as char *argv[]

```
#include<stdio.h>

int main(int argc, char *argv[]) {
   if (argc<2)
      printf ("Too few args!\n");
   else if (argc == 2)
      printf (“Hello %s\n",argv[1]);
   else
      printf("Too many args!\n");
   return 0;
}
```

```
$ ./a.out
Too few args!
```

```
$ ./a.out Subidh
Hello Subidh
$ ./a.out World
Hello World
$ ./a.out IC-100
Hello IC-100
```

```
$ ./a.out Hey There
Too many args!
```

What about Other Types?

- Write a program that takes two numbers (integers) on command line and prints their sum.
- Problem:
  - Everything on command line is read as string!
  - How do I convert string to int?
- Solution: Library functions in stdlib.h
  - atoi: takes a string and converts to int

    atoi("1234") is 1234, atoi("123ab") is 123, atoi("ab") is 0
  - atof: converts a string to double
- Other variations : atol, atoll

Adding 2 Numbers

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char *argv[]) {
   if (argc != 3)
      printf (“Bad args!\n");
   else {
      int a = atoi(argv[1]);
      int b = atoi(argv[2]);
      printf (“%d\n",a+b);
   }
   return 0;
}
```

```
$ ./a.out
Bad args!
```

```
$ ./a.out 3 4
7
$ ./a.out 3 -4
-1
$ ./a.out 3 four
3
```

```
$ ./a.out 3 4 5
Bad args!
```

Command Line Sorting

```
int main(int argc, char *argv[]) {
   int *ar, n;

   n = argc - 1;
   ar = (int *)malloc(sizeof(int) * n);
   for (i=0; i<n; i++)
      ar[i] = atoi(argv[i+1]);

   sort(ar, n);

   for (i=0; i<n; i++)
      printf("%d ",ar[i]);
   return 0;
}
```

```
Void sort (
        int *arr, int
n)
{
    …
}
```

Renaming Executable

```
int main(int argc, char *argv[]) {
   int *ar, n;

   n = argc - 1;
   ar = (int *)malloc(sizeof(int) * n);
   for (i=0; i<n; i++)
      ar[i] = atoi(argv[i+1]);

  sort(ar, n);

   for (i=0; i<n; i++)
      printf("%d ",ar[i]);
   return 0;
}
```

```
void sort (
        int *arr, int
n)
{
    ...
}
```

The flag “-o” of gcc can be used to give user-defined name to the executable, e.g.

$ gcc –o sort myfile.c

$ ./sort 1 4 2
-10 -1 1 2 3 4 5 6 9 10

More Unix Commands

- Listing your file in a directory
- ls [options] [file/directory name]
  - ls
  - ls –l
  - ls –a
  - ls –l –a
  - ls –l ..
  - ls –l .. .

More Unix Commands

- Creating a new directory

  mkdir [options] [directory name]

  - mkdir dir1

- Changing directory

  cd [options] [directory path]

  - cd
  - cd ..
  - cd IC100

More Unix Commands

- Moving a file

  mv [options] [source] [target]
  - mv dir1 dir2
  - mv file1 file2
  - mv file1 dir1
  - mv –i file1 file2
- Copying a file

  cp [options] [source] [target]
- Similar to mv, but the source file remains intact